A Molarium architecture and executable design history

This appendix specifies the implementation beneath the browser interface. It separates deployed capabilities from proposed workflows and closes with the selected SOS1 route underlying the paper movie.

A.1 Execution architecture

WebAssembly and WebGPU give the browser two different kinds of calculator. Emscripten can compile established C and C++ programs into WebAssembly's portable instruction format. WebGPU lets the browser submit parallel arithmetic to the laptop's graphics processor [1–3]. Neither knows chemistry by itself.

Molarium therefore separates chemical interpretation from calculation. RDKit identifies chemical environments in the molecular graph. Molarium's JavaScript parameterizer then applies the OpenFF Sage tables to produce a numerical recipe: atom properties, bonded terms, nonbonded terms, and constraints. Missing bond, angle, torsion, or van der Waals parameters stop preparation rather than being guessed. OpenMM or WebGPU evaluates the same accepted recipe. The interface maps the returned coordinates and energies back onto the original atoms. This separation allows one engine to serve as a direct check on the other.

For readers outside software development, ETKDG proposes plausible starting 3D shapes [4, 5]. SMARTS patterns recognize chemical environments in a molecular graph. MMFF94 and UFF provide quick ligand-only geometry cleanup [6, 7]. OBC2/ACE approximates solvent screening and the nonpolar cost of placing a molecule in solvent. "Analytical forces" means forces obtained directly by differentiating the energy equations, rather than by repeatedly perturbing each coordinate.

The information flow is deliberately narrow:

PDB or SMILES input
↓
molecular graph, stable atom identities, and coordinates
↓
RDKit interpretation → Sage numerical recipe
↓
three calculators:
OpenMM/WebAssembly reference; direct WebGPU; replica WebGPU
↓
viewer + action audit + numerical labbook + design decisions

The architectural claim concerns this composition, not novelty in each of its parts. Background workers, atomwise GPU decomposition, graph-coloured constraints, fixed-point accumulation, canonical JSON, and cryptographic hashes are established techniques. Molarium combines them behind one numerical and action contract, then tests the boundaries created by that combination in a browser. The implementation results below therefore distinguish three things: routine choices needed to reproduce the software, changes forced by observed failures, and measured workload-dependent behaviour.

Table 1 summarizes the concrete implementation. The numerical recipe crossing these layers has a narrow contract:

- **Input.** A molecular graph plus coordinates, masses, charges, bonded terms, nonbonded terms, exceptions, and constraints.

Table 1: Current browser implementation.

Task	Software used	What it does
Interpret the molecule	RDKit 2025.03.4, WebAssembly	SMILES/graphs, ETKDGv3, SMARTS matching, MMFF94, explicit UFF fallback, Gasteiger charges
Assign the force field	OpenFF Sage 2.1.0	Produces the complete numerical recipe: particles, bonds, angles, torsions, nonbonded terms, exceptions, and constraints
Reference calculation	OpenMM 8.2.0 Reference, WebAssembly	High-precision energy/forces, L-BFGS minimization, and short dynamics; used as an answer key
Fast local calculation	WebGPU/WGSL, 32-bit floating point	Evaluates one system directly on the GPU; includes analytical forces, OBC2/ACE, minimization, and short dynamics
Many-copy calculation	STORMM-inspired WebGPU engine	Advances many copies of one topology together; exact integer-based accumulation; current limit 512 atoms per copy
Display and record	JavaScript modules, background workers, Canvas, Web Crypto	Keeps atom identity, executes allowed actions, and writes reproducible JSON records with SHA-256 fingerprints

- **Units.** nm, ps, daltons, kJ/mol, and kJ/mol/nm internally; angstroms and kcal/mol in the interface.
- **Isolation.** Each engine lives in a reusable background worker—a separate browser task—so the molecular canvas and controls can remain responsive. Jobs carry identifiers that match results to requests. A crashed worker is discarded and recreated.
- **Output.** Compact arrays of coordinates, forces, energies, and trajectory frames return to the same molecular object and atom identities.

For ordinary small molecules, a JavaScript SMIRNOFF parameterizer uses RDKit to apply Sage 2.1.0 patterns and assign labelled Gasteiger charges [8, 9]. Prepared Rosemary fixtures keep their exported numerical recipes and NAGL charges. CPU/GPU comparisons therefore use identical graphs and numbers, not two separately prepared approximations.

A.1.1 OpenMM WebAssembly reference

Cross-compiling C++ and exposing a C interface are established deployment techniques. The relevant results here are that the browser build preserved the native calculation and that constrained setup exposed one browser-specific threading failure.

Deployment. We compiled OpenMM 8.2.0 with Emscripten 6.0.6-git and CMake 3.31.0 [10, 11]. A thin C interface copies the numerical recipe into OpenMM and returns energies, forces, or coordinates. The shipped source, two patches, interface, and built files carry recorded fingerprints. A rebuild can be audited against those inputs; byte-identical recompilation is not asserted.

Reference configuration. Nonperiodic OpenMM `System`; double-precision Reference platform; energy, forces, L-BFGS, short dynamics, optional X–H constraints, 2 fs Langevin-middle integration, cutoff, and OBC2/ACE with mbondi2 radii [12].

Browser-specific failure. OpenMM normally creates helper CPU threads while setting up constrained motion. This browser build has no such threads; without a small serial patch, setup waits indefinitely. The patch affects only this WebAssembly build, not native OpenMM.

Measured parity. First, the same C interface compiled natively and for WebAssembly agreed to 2.84×10^{-14} kcal/mol maximum energy difference and 3.76×10^{-15} relative force RMS. This tests whether OpenMM survived conversion to the browser. We then sent five hash-selected protein–ligand poses through browser Sage and OpenMM using the same numerical inputs. The maximum energy difference was 1.24×10^{-4} kcal/mol in vacuum and 2.76×10^{-4} kcal/mol with OBC2. Maximum relative force RMS error was 1.16×10^{-5} in both modes. These fixed-input regression results are not general force-field accuracy claims.

A.1.2 Direct WebGPU mechanics

This path does not run OpenMM on the GPU. It evaluates the same force-field equations directly in WebGPU Shading Language (WGSL), using 32-bit floating-point arithmetic. Its central trade-off is simple: repeat some arithmetic so that each force has exactly one writer. This favours deterministic browser execution over the more cooperative decompositions used by highly tuned GPU molecular-dynamics codes [13].

- **Single-writer force accumulation—implementation choice.** One invocation owns one atom's output force. A compact incidence list gives it only the bonds, angles, and torsions involving that atom. Pair and valence contributions are recomputed where necessary, after which the invocation writes its force once. The method therefore needs no floating-point atomic update to the force buffer; WGSL defines atomic integer types but no atomic floating-point type [3]. One-invocation-per-atom is a conventional GPU decomposition, not a new algorithm.

- **Quadratic pair allocation removed—failure-driven change.** The first representation stored data for every possible atom pair and cost $32N^2$ bytes: approximately 32 MB at 1,000 atoms and 2 GB at 8,000 atoms. The deployed representation stores only excluded or specially scaled pairs; ordinary Lennard–Jones parameters are mixed on demand. This change removed the direct engine's former 512-atom allocation ceiling.

- **OBC2/ACE mapped to three passes—method implementation.** The first pass estimates Born radii, the second propagates their coordinate derivatives, and the third accumulates solvent and nonpolar energies and forces. The pass structure is an implementation of the published model, not a new solvent model.

- **Graph-coloured constraints—implementation choice.** Optional X–H constraints use target distances from the assigned bonds. Constraints with no shared atom are grouped so SHAKE and RATTLE corrections can run concurrently [14, 15]. Seeded Langevin dynamics then supports 1 fs flexible or 2 fs constrained steps.

- **Bounded pocket relaxation—product behaviour.** A bounded steepest-descent procedure cleans geometry. Zero inverse mass fixes outer receptor atoms, leaving only the ligand and selected pocket atoms free.

Two measurements changed or bounded the design. First, a conventional neighbour-list cutoff did not help at these interactive system sizes. With a 0.2 nm Verlet skin rebuilt every 20 steps,

Table 2: Operating limits in the current direct WebGPU worker.

Setting	Current value
Principal force workgroup	64 atomwise calculations
Time step	1 fs flexible; 2 fs with X–H constraints
Constraint solve	Four graph-coloured sweeps by default; at most 32
Minimization	750 iterations by default; at most 2,000; each displacement capped at 0.001 nm
Direct dynamics request	At most 5,000 steps; longer requests use the ensemble route
Validation cutoff	1.0 nm cutoff; 0.2 nm skin; rebuild every 20 steps

it made 5,000 constrained OBC2 Trp-cage steps 46% slower and 1,000 ubiquitin steps 17% slower than the full nonbonded range on an Apple M1 Pro. Molarium therefore retains this route for validation but does not expose it in the product.

Second, GPU benefit depended on amortization. One Trp-cage energy/force call took 25.5 ms in WebGPU and 5.7 ms in OpenMM Reference: dispatch cost exceeded the saved arithmetic. Across 1,000 dynamics steps, WebGPU reached 214.9 versus 102.8 ns/day for 304-atom Trp-cage and 65.18 versus 7.11 ns/day for 1,231-atom ubiquitin. These warmed, nonperiodic, all-pairs measurements identify a workload crossover; they are not claims about complete solvated-protein protocols. The direct path currently excludes periodic boundaries, particle-mesh Ewald electrostatics, explicit solvent, barostats, virtual sites, and arbitrary OpenMM custom forces.

A.1.3 Replica-parallel WebGPU

The second engine targets ensembles of independent copies of one topology. It maps one replica to each GPU workgroup, whereas the direct engine distributes one molecule across atom-owning invocations. This mapping is not itself novel; it creates a different throughput regime and requires a different memory layout. The implementation draws on STORMM's fixed-point and stacked-system design but is independent browser code [16]. Replicas share a topology and parameter layout but never interact.

Fixed-point accumulation is also established rather than a Molarium algorithm: it stores a scaled real number as an integer so concurrent additions can use exact integer operations. Its browser adaptation was consequential because WGSL exposes 32-bit integer atomics. Molarium represents each signed 64-bit accumulator as two 32-bit words and uses 2^{22} counts per kcal/mol for energies, 2^{18} counts per kcal/mol/Å for forces, and 2^{32} counts per Å for coordinates. This paired-word representation replaced 32-bit sums that silently overflowed in clashed systems. It produced bit-for-bit replay for identical seeds on the tested adapter.

The integrator is deliberately conventional: velocity Verlet, seeded Langevin thermalization, and SHAKE/RATTLE. Constraint projection is serial within a replica while replicas remain parallel; the default relative tolerance is 10^{-5} with at most 32 iterations. Commands contain at most 50 steps. The important safety behaviour is fail-closed execution: persistent status words turn non-finite values, clamps, and failed constraints into terminal errors. Current limits are 512 atoms per replica, all-pairs nonbonded work, and the same atom count, topology, and parameters in every copy. Heterogeneous stacked systems remain proposed.

Force evaluation still uses a separate 32-bit floating-point coordinate copy. That copy, not the exact store, sets the remaining spatial precision boundary: translating a test system 500 Å from the origin changed its energies by approximately 3×10^{-4} relative. A future implementation should recenter each work unit; widening the integer store cannot correct arithmetic already performed on

the floating-point copy.

The many-copy design pays off only when enough work is available. In warm tests, OpenMM Reference was 13.3-fold faster for one C16 molecule and 3.5-fold faster for one 27-water system. WebGPU was 10.3-fold faster for 1,024 C16 replicas and 24.1-fold faster for 256 water replicas. These comparisons measure aggregate replica throughput; the two engines do not use identical integrators.

A.2 How the numerical rewrite was accelerated

The speed-up came from a shared executable specification, not from asking an agent to invent molecular dynamics from prose. Every implementation received the same atom order, coordinates, parameters, units, and named energy terms as the OpenMM reference. The agent then used a conventional differential-testing loop:

1. start with exactly the same molecular graph, coordinates, and force-field numbers used by the CPU calculation;
2. implement one term at a time—for example bonds before angles, and angles before torsions—including both the GPU equation and the arrangement of its input numbers in memory;
3. ask OpenMM to evaluate the identical input;
4. compare each named energy term and every x, y, and z force;
5. stop at the first difference, correct it, and save a small test that would fail if the error returned;
6. finally run the real WebGPU and WebAssembly code in a browser, rather than a simplified stand-in.

Differential testing is not new. What shortened this implementation was making the complete path—chemical typing, OpenMM adapter, GPU equations, browser workers, controls, and regression tests—addressable in one workspace. A visible failure could be reduced to the first disagreeing term without translating files or atom identities between teams or tools. Reusing exact inputs reduced coordination; it did not relax the numerical standard.

Responsibility remained divided. Published methods and OpenMM supplied the equations and reference values. The human team set the scientific boundary, chose acceptable workflows, and interpreted results. The coding agent translated terms into browser code, isolated discrepancies, and retained regression tests. The public history shows an iterative implementation, but its first public commit was consolidated; it does not support a precise claim about elapsed development time.

Several failures changed the architecture rather than being hidden by wider tolerances (Table 3):

Table 3: Observed failures that changed the implementation.

Observation	Design change	Retained check
Pair table grew as 32 times atom-count squared	Store only exceptional pairs; mix ordinary pairs as needed	Large-system allocation and parity tests
32-bit force/energy sums overflowed	Paired-word 64-bit fixed-point accumulation and fault flags	Clashed system at about 2 million kcal/mol
Coordinates wrapped beyond 128 Å	64-bit fixed-point coordinate store plus floating-point working copy	Whole system translated to 500 Å
Standard cutoff ran slower	Keep it for validation; omit it from product controls	Fixed Trp-cage/ubiquitin timing protocols
Apparent engine mismatch	Check atom order, hashes, and solvent mode before changing code	Native/WebAssembly parity and matched-solvent comparison

The retained tests expose the seams where implementation errors actually appeared:

- **Equation seam:** minimal bonds, angles, torsions, and exceptions catch formula, sign, unit, and periodicity errors.
- **Representation seam:** clashes, constraints, overflow, and translated coordinates exercise projection and numerical range.
- **Engine seam:** identical OpenMM and WebGPU inputs identify the first disagreeing energy component or force coordinate.
- **System seam:** protein fixtures exercise memory layout, special pairs, and transfer between workers and the molecular object.
- **Browser seam:** the shipped GPU and WebAssembly workers exercise adapter support, worker recovery, and the visible controls.
- **Performance seam:** fixed hardware and protocols retain correct but slower results, including the rejected cutoff design.

Fixed random seeds and stable atom IDs make failures repeatable. Both engines reject impossible inputs and invalid memory ranges before display. The ensemble engine also stops on fixed-point overflow warnings or unconverged constraints; the direct engine reports its final constraint error, which browser tests compare with an acceptance threshold. Curated debugging records connect each observed failure to its diagnosis, code change, protective test, and remaining uncertainty; private raw transcripts are not scientific evidence.

Reproduction difficulty and portability boundary. Reproducing the present fixed-topology, nonperiodic mechanics was tractable once the reference calculation became executable, but it was not automatic. The difficulty was uneven:

- **Individual force-field terms were bounded problems.** Each bond, angle, torsion, pair, or solvent contribution had a published equation, a fixed input, and an OpenMM result against which to compare it. Analytical derivatives and OBC2 coupling still required careful work, but discrepancies could be localized.

- **The difficult problems crossed layers.** Atom identities had to survive chemical perception, parameterization, GPU packing, calculation, and display. Force ownership had to avoid races. Numerical ranges had to survive clashes and translations. Long calculations had to yield often enough that the browser remained usable.
- **Validation remained irreducible.** An agent could translate and test each term quickly, but it could not infer that two runs represented the same physical model. Atom order, solvent mode, cutoffs, constraints, units, and charge provenance still had to be matched explicitly.

These results do not imply that any simulation algorithm can be copied into WebGPU. They support a narrower working hypothesis: WebGPU is a strong target when state fits regular arrays, work can be divided into local or pairwise operations, global dependencies can be separated into a small number of kernel passes, and repeated work amortizes dispatch. Fixed-topology mechanics, local minimization, independent replicas, and graph-colourable constraints meet those conditions.

Other methods may be expressible but unattractive. Algorithms that depend on 64-bit floating-point arithmetic, frequent device-wide synchronization, rapidly changing graphs, large adaptive sparse structures, or mature FFT and sparse linear-algebra libraries require substantial redesign under current WGSL [3]. Particle-mesh Ewald electrostatics, heterogeneous reactive systems, and some long-timescale production protocols therefore remain outside the demonstrated boundary. For such methods, a WebAssembly reference, a hybrid CPU/GPU path, or a remote specialist engine may be the more faithful design.

A successful rewrite should consequently mean agreement on the same accepted inputs within a declared tolerance, together with measured usefulness on the target workload. It does not mean identical source code, universal bitwise reproducibility, or automatic speed-up.

A.3 Workflows

"Local calculation" and "offline operation" are different claims. In normal connected mode, computation stays on the device, but an explicit PDB/CCD lookup or sequence-alignment search contacts the service named in the interface. Local Lab serves a reviewed checkout from localhost under a restrictive browser Content Security Policy: external structure and alignment controls are disabled, and an egress-canary test fails if a synthetic private-data marker reaches an external request. A hosted page can still change the code it serves on a later visit; the strongest boundary is a reviewed versioned checkout with locally pinned assets.

Supported now.

- **Local small-molecule work:** construct in synchronized 2D/3D; validate the graph; run RDKit or direct-WebGPU calculations; inspect aligned trajectories and energies; export. OpenMM WebAssembly supports internal validation and fixed-scaffold operations but is not a selectable Run-panel method.
- **Protein–ligand pose propagation:** capture a trusted pose; preserve atom lineage; sample only changed ligand branches; impose optional contact hypotheses; and apply a selected pose while the receptor remains fixed. A separate experimental induced-fit action releases the ligand and every atom—backbone included—of protein residues entering a 6 Å pocket shell; all outer atoms remain fixed.
- **Agent-operated design:** call the versioned Chemist Actions API, not arbitrary page code. Here the API is a fixed menu of permitted actions; the agent receives the same state-changing

verbs a chemist can invoke—select, edit, search, apply, relax, and undo—plus read-only inspection. It cannot silently replace coordinates or call arbitrary page functions. Requests run one at a time and record sequence, timestamps, arguments, status, and duration.

Proposed.

- **Local-to-specialist escalation:** export a checkpoint identified by a content fingerprint, together with an explicit protocol for periodic solvent, long trajectories, free energy, or electronic structure; attach returned results as evidence without replacing the local branch. Transport, scheduling, signatures, and trust policy are not implemented.
- **Collaborative molecular review:** branch by chemotype, protonation state, pose, or author; merge by recording decision and parentage, never by averaging coordinates; cite the exact branch and replay.

A.4 Git-like history for molecules and decisions

The Git analogy is simple: a *snapshot* is one molecular state; a *commit* records that state and why it was saved; a *branch* preserves an alternative design route. A merge records which route was chosen and why. It never averages two molecular structures. This is a custom content-addressed ledger inspired by Git, not a Git repository or a general three-way molecular merge algorithm. Three linked record types answer different provenance questions:

Current product boundary. The live application currently keeps at most 500 action records in memory. Clearing the scene clears that audit. Durability therefore requires an explicit *Export API moves* or *Export replay log* download; there is no hidden server sync or automatic molecular version control. The append-only ledger, commit, branch, decision, and verification functions exist in the repository and drive standalone builders and reviews, but the main interface does not yet turn a live session into ledger commits with one click.

Table 4: Separation of operational, numerical, and design provenance.

Record	Question	Contents
Chemist Actions audit	What was requested?	Public action, arguments, timing, status, compact after-state
Calculation labbook	What numerical protocol ran?	Inputs, method, parameters, outputs, hash-linked events
Design campaign	Why did a state advance or stop?	Snapshots, branches, parents, hypotheses, evidence, decisions

Canonical JSON, SHA-256 fingerprints, parent links, and hash-chained events are standard provenance mechanisms. Molarium's contribution is their molecular binding: an operational record states what was requested, a numerical record states what calculation ran, and a design record states why one molecular state advanced. Keeping these questions separate prevents a successful replay from being mistaken for evidence that a design decision was scientifically sound.

The ledger implementation uses schema `molarium.design-campaign/v1` and stores plain JSON in a single standardized ordering (canonical JSON). SHA-256 converts each snapshot, action script, or commit into a 64-character fingerprint: changing even one recorded value changes the fingerprint. Each commit names its snapshot, parent(s), branch, message, optional action script, hypotheses,

evidence, and tags. Events are append-only and each includes the previous event's fingerprint, forming a chain. They also identify the actor (human, agent, system, or import) and source. Decision states are `progressed`, `not-progressed`, `deferred`, `failed`, `duplicate`, `superseded`, and `archived`. Finalization appends a completion event, hashes the campaign, and makes later additions or edited records fail verification. It does not make a JSON file physically unwritable. Hashes detect alteration; they are not author signatures.

Registered design routes and campaign ledgers use separate schemas. The SOS1, BCL-xL, and CDK2 input protocols use `molarium.registered-design-route/v1`; they define the coordinate boundary, hash-pinned hit, and ordered graph edits. Append-only histories use `molarium.design-campaign/v1`; they contain snapshots, commits, branches, events, and decisions. The application validates a route before loading it, and cross-schema tests reject a route as a ledger or a ledger as a route. Frozen predictions made before the schema split retain their original input hash. A migration record stores both hashes and is accepted only when replacing the schema identifier reconstructs the exact frozen bytes; any other field change still fails verification. Public route actions are `designRoute.load`, `designRoute.applyStep`, and `designRoute.inspect`; they take a `routeId`. No compatibility alias is exposed. Campaign-ledger operations remain a separate concern.

The fingerprint seals one exact recorded representation. Two chemically equivalent SMILES, tautomers, protonation states, or atom orderings can still produce different fingerprints. It proves record identity—not chemical equivalence, authorship, or truth.

A decision points to a molecular commit; that commit points to its selected snapshot, parent commits, action script, hypotheses, and evidence. The following field-level sketch shows the linkage. Full identifiers are SHA-256-derived strings rather than the shortened labels shown here.

```
{
  "moleculeCommit": {
    "snapshotId": "snapshot:<selected-child>",
    "parents": ["commit:<parent>"],
    "actionScriptId": "script:<designer-moves>",
    "hypothesisIds": ["event:<pocket-must-open>"]
  },
  "decisionEvent": {
    "kind": "decision.recorded",
    "actorId": "chemist.01",
    "payload": {
      "targetCommitId": "commit:<selected-child>",
      "disposition": "progressed",
      "reasonCodes": ["contact-loss"],
      "rationale": "Phe-out removes growth-induced clashes.",
      "evidenceIds": ["event:<rotamer-search>"]
    }
  }
}
```

The following is an illustrative excerpt, not a complete executable route. The full story also contains preparation, reference capture, molecular growth, pose search, parameterization, and relaxation in the required order. A replay is a list of allowed instructions, not a program:

```
{
  "schema": "molarium.chemist-action-script/v1",
  "label": "SOS1 selected Phe890 route",
  "actions": [
    {
      "action": "designRoute.load",
      "args": {"routeId": "sos1-hit-only"},
      "caption": "Load the coordinate-bearing 5OVE/AXE hit only"
    },
    {
      "action": "pose.applySidechainRotamer",
      "args": {"index": 5},
      "caption": "Apply selected Phe890 rotamer index 5"
    }
  ]
}
```

Replay rules:

- schema `molarium.chemist-action-script/v1`; standardized JSON; unknown actions, executable code, direct coordinate replacement, and unreasonably large or deeply nested inputs are rejected;
- imported files are capped at 2 MiB; one action's arguments are capped at 32 KiB, eight nested levels, and 2,048 values;
- an action may give a name to a returned value, such as a stable atom ID; a later action may reuse that name without embedding new code;
- the runner executes one allowed action at a time through `api.execute({action,args})`, assigns a repeatable request ID from the script fingerprint, and stops at the first failure;
- result schema `molarium.chemist-action-replay/v1` records the source fingerprint and every step outcome;
- import starts on a blank canvas at move zero; Play advances, Pause stops at the next public-action boundary, and Restart returns to blank;
- presentation actions may focus or style the view. They cannot change the molecular graph or coordinates except through the same allowed actions.

A stable URL such as https://molarium.org/sos1-hit-to-bay293 identifies the story. The friendly URL may point to a newer release later; the release commit and selected-route file fingerprint `c3c5cfff681ab34c...a94f4dbf2021` identify the exact bytes reviewed. Its canonical action-script fingerprint `24b6a50718c7cdd3...8fb896fc9` identifies the normalized JSON content, so harmless whitespace changes do not create a logically new route.

The practical workflow is therefore: capture a trusted starting snapshot; explore poses or conformations while saving the action audit; apply and relax a selected state while saving its numerical record; record the decision and parentage in the ledger; and publish the replay script, result, release commit, and fingerprints together. The first three steps run in the live product. The one-click conversion of those exports into the final ledger remains to be wired into that interface.

A.5 SOS1 hit-to-BAY-293 interface movie

The movie replays the selected SOS1 route from the series reported by Hillig and colleagues [17]. The published analogue graphs for AWT, AWZ, AWW, and AXH were supplied. Their poses and receptor conformations were not. Thus the blinded question was where each known analogue would sit and whether its growth required the pocket to reorganize—not which compound to invent, whether to synthesize it, or what potency it would have. The boundary was:

- the sole coordinate-bearing design input was 5OVE/AXE;
- each analogue began from the preceding frozen prediction, never from the corresponding later crystal;
- the selected route contains 33 recorded API actions: 29 molecular or protocol operations and four workspace switches;
- the complete exploration audit contains 89 completed API calls, including three Phe890/pose branches, nine undo calls, and deterministic reselection; these are operational records, not formal ledger decisions;
- the interface presentation adds 15 display-only actions, for 48 replay steps in the film;
- later crystals: opened only after four prediction checkpoints and the audit were hashed;
- display and focus actions could not change the graph or coordinates.

These are substantial graph rewrites rather than a sequence of single-atom substitutions. Table 5 counts the atom lineage retained across each change. The first transition also had four possible graph mappings; the later transitions each had one.

Table 5: Heavy-atom lineage across the supplied SOS1 analogue graphs.

Transition	Retained	Deleted	Added	Candidate mappings
AXE–AWT	23	4	6	4
AWT–AWZ	19	10	12	1
AWZ–AWW	16	15	15	1
AWW–AXH	15	16	17	1

"64 search chains" means 64 independently seeded local pose searches, not 64 protein chains. "Parameterize without motion" means assign coefficients while leaving coordinates unchanged; only the following relaxation may move atoms. The experimental SOS1 route used whole-system Sage 2.1.0/Gasteiger preparation, not the prepared Rosemary fixtures. AWT, AWZ, AWW, and AXH contained 7,929, 7,933, 7,935, and 7,937 particles, respectively. This charge-model boundary is part of the method, not a standard Sage AM1-BCC or NAGL preparation.

Before flexible relaxation, the application stores the ligand bond lengths. It restores the previous pose if any heavy-atom bond is both more than 0.45 Å longer and 1.25-fold its assigned target, or both more than 0.35 Å shorter and below 0.75-fold its target. A lower pose score therefore cannot buy a broken molecule.

Table 6: Scientific beats in the SOS1 replay. Predictions, not future crystals, seed the next step.

Beat	Visible action	State/evidence
0	Load and prepare	5OVE/AXE only; capture compound 1 reference and hit boundary
1	Rewrite to compound 17 (AWT)	Run 64 independent reference-guided pose searches; apply the selected pose; parameterize; relax
2	Merge to compound 18 (AWZ)	Propagate from predicted AWT, not 5OVF; apply, parameterize, relax
3	Grow compound 21 (AWW)	New ring enters the Phe890-in volume; the fixed-core representative gains four clashes
4	Resolve the pocket	Enumerate discrete Phe890 rotamers; select the outward branch; accept its moved receptor as the next reference
5	Predict BAY-293 (AXH)	Grow from frozen AWW; 64-chain pose search; apply, parameterize without motion, relax, freeze
6	Reveal holdouts	Compare AWT/AWZ/AWW/BAY-293 with 5OVF/5OVG/5OVH/5OVI

The conformational step is necessary because local minimization normally cannot cross the torsional barrier between side-chain rotamers. Molarium therefore created separate starting basins before local ligand–pocket relaxation. It ranked feasible branches first by the fewest clashes added by growth, then by pose score and finite relaxed energy. The nominal $-60°$ branch added four clashes above the fixed-core baseline and was infeasible; the $-180°$ branch added none and was selected; the $+60°$ branch added two and remained a feasible alternative. This is the causal point of the story: the analogue's growth made the inward state untenable before 5OVH or 5OVI was opened.

For structural evaluation, each predicted receptor was aligned to its holdout on 23, 25, 25, and 27 pocket Cα atoms for AWT through AXH. Ligand RMSD was then computed over 29, 31, 31, and 32 heavy atoms, minimizing over 2, 1, 1, and 1 graph-symmetry mappings, respectively. The ligand itself was not fitted.

Table 7: Post-freeze structural evaluation. Torsions are predicted/crystal in degrees; ligand RMSD is receptor-aligned and graph-symmetry-minimized.

Prediction	Holdout	Ligand RMSD (Å)	Phe890 chi1	Phe890 chi2
AWT	5OVF	0.516	-77.6 / -71.5	-71.7 / -71.7
AWZ	5OVG	1.528	-73.4 / -75.6	-92.0 / -87.9
AWW	5OVH	2.137	-172.2 / +175.8	+116.1 / +76.3
BAY-293	5OVI	1.215	-172.4 / -166.5	+108.7 / +71.7

The ligand poses are useful but not atom-perfect. The central success is the prospective pocket switch. The replay recovered the experimentally observed outward χ_1 basin before seeing 5OVH or 5OVI; it did not recover the exact χ_2 terminal-ring orientation, which differs by approximately $40°$ for AWW and $37°$ for BAY-293.

The interface film starts on a genuinely blank canvas, selects the same JSON available to readers through *Designer moves*, and presses the visible Play control. Subsequent operations run through the public Chemist Actions API while the corresponding visible control is highlighted; the renderer does not literally click every individual control. Unused panels collapse. Candidate generation holds its result list while coordinates remain frozen, and only Apply changes the molecular state.

The film shows discrete before/after molecular states. It does not interpolate coordinates or

present the transition as molecular dynamics. Result frames are held for roughly 0.9–3.4 s so a reader can inspect new information. The current 62.58-s interface cut condenses approximately 908.5 s of durations in its 48-action audit; full operation durations remain in the audit and render manifest. The film is therefore an explanatory replay, not a runtime benchmark.

The current replay code can focus on heavy atoms displaced by at least 0.08 Å, highlight up to the 24 largest changes, and retain their local context. The existing complete MP4 predates that changed-region mode and uses component-level focus. It must be regenerated before submission if the paper claims that red changed-atom emphasis appears in the delivered cut.

Tyr884 is separate historical context. Comparing fragment-bound 6EPM with 5OVE shows that the inhibitory Tyr884 arrangement already exists in the starting 5OVE coordinates. The hit-only route predicts no Tyr884 movement.

Recommended main-text frames are: (1) blank imported story; (2) 5OVE hit boundary; (3) AWW/Phe890-in clash; (4) applied Phe890-out branch; (5) frozen BAY-293 prediction; and (6) separately labelled post-freeze 5OVI comparison.

References

[1] Andreas Haas, Andreas Rossberg, Derek L. Schuff, Ben L. Titzer, Michael Holman, Dan Gohman, Luke Wagner, Alon Zakai, and JF Bastien. Bringing the web up to speed with WebAssembly. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 185–200, 2017. doi: 10.1145/3062341.3062363.

[2] W3C GPU for the Web Working Group. WebGPU. https://www.w3.org/TR/webgpu/, 2025. Accessed 2026-08-16.

[3] W3C GPU for the Web Working Group. WebGPU Shading Language. https://www.w3.org/TR/WGSL/, 2026. Accessed 2026-09-02.

[4] Sereina Riniker and Gregory A. Landrum. Better informed distance geometry: Using what we know to improve conformation generation. *Journal of Chemical Information and Modeling*, 55(12):2562–2574, 2015. doi: 10.1021/acs.jcim.5b00654.

[5] Shuzhe Wang, Jagna Witek, Gregory A. Landrum, and Sereina Riniker. Improving conformer generation for small rings and macrocycles based on distance geometry and experimental torsional-angle preferences. *Journal of Chemical Information and Modeling*, 60(4):2044–2058, 2020. doi: 10.1021/acs.jcim.0c00025.

[6] Thomas A. Halgren. Merck molecular force field. I. basis, form, scope, parameterization, and performance of MMFF94. *Journal of Computational Chemistry*, 17(5–6):490–519, 1996. doi: 10.1002/(SICI)1096-987X(199604)17:5/6<490::AID-JCC1>3.0.CO;2-P.

[7] Anthony K. Rappé, Colleen J. Casewit, K. S. Colwell, William A. Goddard, and W. M. Skiff. UFF, a full periodic table force field for molecular mechanics and molecular dynamics simulations. *Journal of the American Chemical Society*, 114(25):10024–10035, 1992. doi: 10.1021/ja00051a040.

[8] Simon Boothroyd, Pavan Kumar Behara, Oliver C. Madin, David F. Hahn, Hiromi Jang, Vytautas Gapsys, Jeffrey R. Wagner, Josh T. Horton, David L. Dotson, Matthew W. Thompson, Jessica Maat, Trevor Gokey, Lee-Ping Wang, Daniel J. Cole, Michael K. Gilson, John D. Chodera, Christopher I. Bayly, Michael R. Shirts, and David L. Mobley. Development and benchmarking of Open Force Field 2.0.0: The Sage small molecule force field. *Journal of Chemical Theory and Computation*, 19(11):3251–3275, 2023. doi: 10.1021/acs.jctc.3c00039.

[9] Johann Gasteiger and Mario Marsili. Iterative partial equalization of orbital electronegativity—a rapid access to atomic charges. *Tetrahedron*, 36(22):3219–3228, 1980. doi: 10.1016/0040-4020(80)80168-2.

[10] Peter Eastman, Jason Swails, John D. Chodera, Robert T. McGibbon, Yutong Zhao, Kyle A. Beauchamp, Lee-Ping Wang, Andrew C. Simmonett, Matthew P. Harrigan, Chaya D. Stern, Rafal P. Wiewiora, Bernard R. Brooks, and Vijay S. Pande. OpenMM 7: Rapid development of high performance algorithms for molecular dynamics. *PLOS Computational Biology*, 13(7):e1005659, 2017. doi: 10.1371/journal.pcbi.1005659.

[11] Peter Eastman, Raimondas Galvelis, Raúl P. Peláez, Charlles R. A. Abreu, Stephen E. Farr, Emilio Gallicchio, Anton Gorenko, Michael M. Henry, Frank Hu, Jing Huang, et al. OpenMM 8: Molecular dynamics simulation with machine learning potentials. *The Journal of Physical Chemistry B*, 128(1): 109–116, 2024. doi: 10.1021/acs.jpcb.3c06662.

[12] Alexey Onufriev, Donald Bashford, and David A. Case. Exploring protein native states and large-scale conformational changes with a modified generalized born model. *Proteins: Structure, Function, and Bioinformatics*, 55(2):383–394, 2004. doi: 10.1002/prot.20033.

[13] Jens Glaser, Trung Dac Nguyen, Joshua A. Anderson, Pak Lui, Filippo Spiga, Jaime A. Millan, David C. Morse, and Sharon C. Glotzer. Strong scaling of general-purpose molecular dynamics simulations on GPUs. *Computer Physics Communications*, 192:97–107, 2015. doi: 10.1016/j.cpc.2015.02.028.

[14] Jean-Paul Ryckaert, Giovanni Ciccotti, and Herman J. C. Berendsen. Numerical integration of the cartesian equations of motion of a system with constraints: Molecular dynamics of n-alkanes. *Journal of Computational Physics*, 23(3):327–341, 1977. doi: 10.1016/0021-9991(77)90098-5.

[15] Hans C. Andersen. RATTLE: A "velocity" version of the SHAKE algorithm for molecular dynamics calculations. *Journal of Computational Physics*, 52(1):24–34, 1983. doi: 10.1016/0021-9991(83)90014-1.

[16] David S. Cerutti, Rafal Wiewiora, Simon Boothroyd, and Woody Sherman. STORMM: Structure and topology replica molecular mechanics for chemical simulations. *The Journal of Chemical Physics*, 161 (3):032501, 2024. doi: 10.1063/5.0211032.

[17] Roman C. Hillig, Brice Sautier, Jens Schroeder, Dieter Moosmayer, André Hilpmann, Christian M. Stegmann, Nicolas D. Werbeck, Hans Briem, Ulf Boemer, Joerg Weiske, Volker Badock, Julia Mastouri, Kirstin Petersen, Gerhard Siemeister, Jan D. Kahmann, Dennis Wegener, Niels Böhnke, Knut Eis, Keith Graham, Lars Wortmann, Franz von Nussbaum, and Benjamin Bader. Discovery of potent SOS1 inhibitors that block RAS activation via disruption of the RAS–SOS1 interaction. *Proceedings of the National Academy of Sciences of the United States of America*, 116(7):2551–2560, 2019. doi: 10.1073/pnas.1812963116.